Task 05: Examine System Calls

Operating Systems (CSA04) — Assignment 1 (CO1)

Student Name: Rohit L
Register Number: 192511236
Course: Operating Systems (CSA04)
Marks: 4 Marks (Bloom's: K5 - Evaluate)
GitHub: `https://github.com/rohitaju`
Sub-directory: `Task05/`

> ***Task 5 Description:*** *Examine system call categories. Explain step-by-step what happens when a program calls read(): library call, trap, mode switch, kernel execution, and return. Discuss parameter passing methods. Analyze strace outputs and compare buffered C library calls (fread) vs. raw system calls (read).*

1. SYSTEM CALL CATEGORIES & EXAMPLES

Category	POSIX System Calls	Hospital System Application
Process Control	`fork()`, `exec()`, `waitpid()`, `exit()`	Spawning query-handling worker processes on the central database server.
File Management	`open()`, `read()`, `write()`, `close()`	Reading and persisting patient health records and configuration profiles.
Device Management	`ioctl()`, `read()`, `write()`	Directly configuring SpO2 sensor sample rates and reading ECG hardware registers.
Information	`gettimeofday()`, `clock_gettime()`	High-precision millisecond timestamping of physiological alarm conditions.
Communication	`socket()`, `bind()`, `connect()`, `send()`	Transmitting telemetry packets to central nursing monitors over local networks.
Protection	`chmod()`, `chown()`, `setuid()`	Restricting access to medical records based on authenticated role permissions.

2. STEP-BY-STEP MECHANICS OF `READ()`

1. **User Invocation:** Application calls `read(fd, buffer, count)` via the C runtime library (glibc).
2. **Register Marshalling (x86_64):** Arguments are loaded into registers: `rax = 0` (syscall number for read), `rdi = fd`, `rsi = buffer`, `rdx = count`.
3. **Hardware Trap:** CPU executes `syscall`. Hardware saves user `rip` to `rcx`, flags to `r11`, sets the mode bit to Ring 0 (Kernel Mode), and branches to the kernel system call entry point.
4. **Kernel Execution:** Kernel pushes register state to the per-process kernel stack, indexes the dispatch table, and runs `sys_read()`. The VFS layer verifies memory permissions and invokes the device driver or page cache.
5. **Return Transition:** The return value is placed in `rax`. Kernel restores registers and executes `sysretq`, returning the CPU to Ring 3 (User Mode).

3. ALTERNATIVE CONCEPT: C LIBRARY BUFFERING VS. DIRECT SYSTEM CALLS

For reading an `n`-byte file sequentially:

- **Direct System Calls (`read` with 1-byte buffer):** Triggers `n` distinct kernel transitions: `Calls = n`. Overhead is $O(n \times T_{trap})$. For 10,000 bytes, this requires 10,001 system calls.
- **C Library Buffering (`fread`):** Allocates an internal 4,096-byte buffer in user space. It fetches 4,096 bytes per system call and serves subsequent requests directly from user RAM: $Calls = \lceil n / 4096 \rceil + 1$. For 10,000 bytes, this requires only 4 system calls—a **99.96% reduction in kernel transitions** with minimal memory overhead (4 KB).

4. EMPIRICAL `STRACE` VERIFICATION

```
$ strace -c ./task06_copy
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 94.21    0.003810           0     10001           read
  5.79    0.000234           0         4           write
------ ----------- ----------- --------- --------- ----------------
100.00    0.004044                 10005           total
```